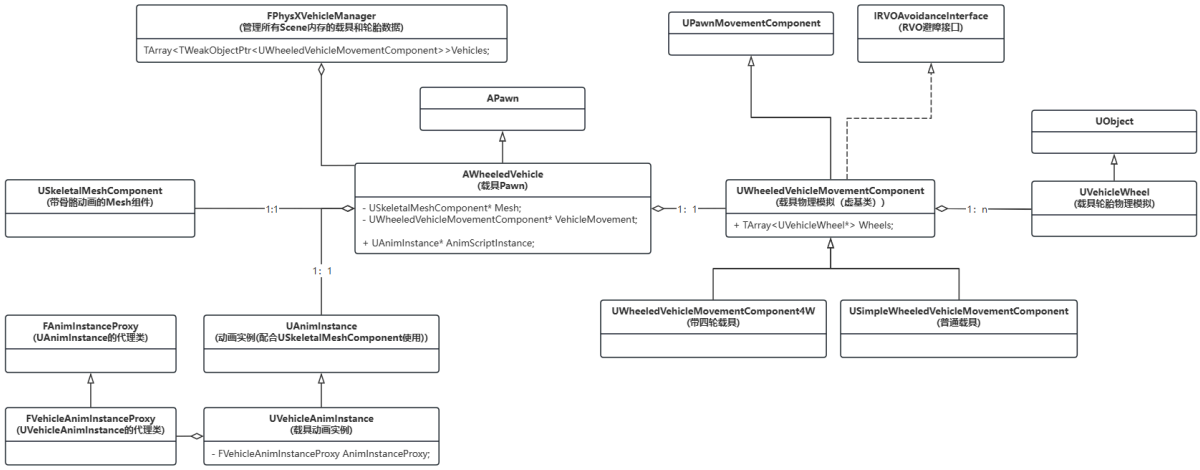
Vehicle 引擎代码概览

一、Class



> PhysX版本

- FPhysXVehicleManager
- AWheeledVehicle
- UWheeledVehicleMovementComponent
- UWheeledVehicleMovementComponent4W
- UVehicleWheel
- IRVOAvoidanceInterface
- FAnimInstanceProxy
- FVehicleAnimInstance

二、Tick

2.1 TickGroup

C/C++

```
UWorld::Tick
{
    ...

    // Actually tick actors now that context is set up
    SetupPhysicsTickFunctions(DeltaSeconds);
    TickGroup = TG_PrePhysics; // reset this to the start tick group
    FTickTaskManagerInterface::Get().StartFrame(this, DeltaSeconds, TickType, LevelsToTick);

    SCOPE_CYCLE_COUNTER(STAT_TickTime);
    {
        SCOPE_TIME_GUARD_MS(TEXT("UWorld::Tick - TG_PrePhysics"), 10);
        SCOPE_CYCLE_COUNTER(STAT_TG_PrePhysics);
        CSV_SCOPED_TIMING_STAT_EXCLUSIVE(PrePhysicsMisc);
        CSV_SCOPED_SET_WAIT_STAT(PrePhysics);
        RunTickGroup(TG_PrePhysics);
    }
    bInTick = false;
    EnsureCollisionTreeIsBuilt();
    bInTick = true;
    {
        SCOPE_CYCLE_COUNTER(STAT_TG_StartPhysics);
        SCOPE_TIME_GUARD_MS(TEXT("UWorld::Tick - TG_StartPhysics"), 10);
        CSV_SCOPED_TIMING_STAT_EXCLUSIVE(StartPhysicsMisc);
        CSV_SCOPED_SET_WAIT_STAT(StartPhysics);
        RunTickGroup(TG_StartPhysics);
    }
    {
        SCOPE_CYCLE_COUNTER(STAT_TG_DuringPhysics);
        SCOPE_TIME_GUARD_MS(TEXT("UWorld::Tick - TG_DuringPhysics"), 10);
        CSV_SCOPED_TIMING_STAT_EXCLUSIVE(DuringPhysicsMisc);
        CSV_SCOPED_SET_WAIT_STAT(DuringPhysics);
        RunTickGroup(TG_DuringPhysics, false); // No wait here, we should run until idle though. We don't care if all of the async ticks are done before we start running post-phys stuff
```

```
    }
    TickGroup = TG_EndPhysics; // set this here so the current tick group is correct during collision notifies, though I am not sure it matters. 'cause of the false up there^^^
    {
        SCOPE_CYCLE_COUNTER(STAT_TG_EndPhysics);
        SCOPE_TIME_GUARD_MS(TEXT("UWorld::Tick - TG_EndPhysics"), 10);
        CSV_SCOPED_TIMING_STAT_EXCLUSIVE(EndPhysicsMisc);
        CSV_SCOPED_SET_WAIT_STAT(EndPhysics);
        RunTickGroup(TG_EndPhysics);
    }
    {
        SCOPE_CYCLE_COUNTER(STAT_TG_PostPhysics);
        SCOPE_TIME_GUARD_MS(TEXT("UWorld::Tick - TG_PostPhysics"), 10);
        CSV_SCOPED_TIMING_STAT_EXCLUSIVE(PostPhysicsMisc);
        CSV_SCOPED_SET_WAIT_STAT(PostPhysics);
        RunTickGroup(TG_PostPhysics);
    }
    ...
}
```

2.2 TG_PrePhysics

这个阶段处理物理模拟开始之前的任务

例如 Pawn 的输入

2.3 TG_StartPhysics

这个阶段处理物理模拟开始时处理的任务

例如物理输入数据，为下一阶段物理模拟做准备

2.4 TG_DuringPhysics

这个阶段处理物理模拟的任务，也就是物理引擎（PhysX/Chaos）层处理物理模拟的时刻，除此之外还有引擎底层其他允许在物理模拟阶段执行的 tick

2.5 TG_EndPhysics

这个阶段处理物理模拟结束时相关的任务

例如 SkeletalMemshComponent 应用物理效果

2.6 TG_PostPhysics

这个阶段处理任何需要保证刚体和布料模拟结束后执行的任务

三、Input

C/C++

```
void AVehicleTestPawn::SetupPlayerInputComponent(class UInputComponent * PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);

    // set up gameplay key bindings
    check(PlayerInputComponent);

    PlayerInputComponent->BindAxis("MoveForward", this, &AVehicleTestPawn::MoveForward);
    PlayerInputComponent->BindAxis("MoveRight", this, &AVehicleTestPawn::MoveRight);
    PlayerInputComponent->BindAxis("LookUp");
    PlayerInputComponent->BindAxis("LookRight");

    PlayerInputComponent->BindAction("Handbrake", IE_Pressed, this, &AVehicleTestPawn::OnHandbrakePressed);
    PlayerInputComponent->BindAction("Handbrake", IE_Released, this, &AVehicleTestPawn::OnHandbrakeReleased);
    PlayerInputComponent->BindAction("SwitchCamera", IE_Pressed, this, &AVehicleTestPawn::OnToggleCamera);

    PlayerInputComponent->BindAction("ResetVR", IE_Pressed, this, &AVehicleTestPawn::OnResetVR);
}
```

TG_PrePhysics 阶段接受 Player 的键盘输入，例如 Throttle（油门）、Steering（方向盘）、Handbrake（手刹）、Brake（刹车）等

四、OnCreatePhysicsState

创建载具物理状态

```
UWheeledVehicleMovementComponent::OnCreatePhysicsState() WheeledVehicleMovementComponent.
UActorComponent::CreatePhysicsState(bool) ActorComponent.cpp:1477
UActorComponent::ExecuteRegisterEvents(FRegisterComponentContext *) ActorComponent.cpp:1537
UActorComponent::RegisterComponentWithWorld(UWorld *, FRegisterComponentContext *) ActorCom
AActor::IncrementalRegisterComponents(int, FRegisterComponentContext *) Actor.cpp:4667
AActor::RegisterAllComponents() Actor.cpp:4566
AActor::PostSpawnInitialize(const FTransform &, AActor *, APawn *, bool, bool, bool) Actor.cpp:3164
UWorld::SpawnActor(UClass *, const FTransform *, const FActorSpawnParameters &) LevelActor.cpp:636
AGameModeBase::SpawnDefaultPawnAtTransform_Implementation(AController *, const FTransform &
```

C/C++

```
void UWheeledVehicleMovementComponent::OnCreatePhysicsState()
{
    Super::OnCreatePhysicsState();

    VehicleSetupTag = FPhysXVehicleManager::VehicleSetupTag;

    // only create physx vehicle in game
    UWorld* World = GetWorld();
    if ( World->IsGameWorld() )
    {
        FPhysScene* PhysScene = World->GetPhysicsScene();

        if ( PhysScene && FPhysXVehicleManager::GetVehicleManagerFromScene(PhysScene) )
        {
            FixupSkeletalMesh();
            CreateVehicle();

            if ( PVehicle )
            {
                FPhysXVehicleManager* VehicleManager = FPhysXVehicleManager::GetVehicleManagerFromScene(PhysScene);
                VehicleManager->AddVehicle( this );

                CreateWheels();

                //LogVehicleSettings( PVehicle );
                SCOPED_SCENE_WRITE_LOCK(VehicleManager->GetScene());
                PVehicle->getRigidDynamicActor()->wakeUp();

                // Need to bind to the notify delegate on the mesh incase physics state is changed
                if(USkeletalMeshComponent* MeshComp = Cast<USkeletalMeshComponent>(GetMesh()))
                {
                    MeshOnPhysicsStateChangeHandle = MeshComp->RegisterOnPhysicsCreatedDelegate(FOnSkelMeshPhysicsCreated::CreateUObject(this, &UWheeledVehicleMovementComponent::RecreatePhysicsState));
```

```
                    if(UVehicleAnimInstance* VehicleAnimInstance = Cast<UVehicleAnimInstance>(MeshComp->GetAnimInstance()))
                    {
                        VehicleAnimInstance->SetWheeledVehicleMovementComponent(this);
                    }
                }
            }
        }
    }
}
```

4.1 CreateVehicle

创建载具

C/C++

```cpp
void UWheeledVehicleMovementComponent::CreateVehicle()
{
    ComputeConstants();

#if WITH_PHYSX_VEHICLES
    if ( PVehicle == NULL )
    {
        if ( CanCreateVehicle() )
        {
            check(UpdatedComponent);
            if (ensure(UpdatedPrimitive != nullptr))
            {
                ensure(UpdatedPrimitive->GetBodyInstance()->IsDynamic());

                SetupVehicle();

                if ( PVehicle != NULL )
                {
                    PostSetupVehicle();
                }
            }
        }
    }
#endif
}

void UWheeledVehicleMovementComponent::SetupVehicle()
{
    if (!UpdatedPrimitive)
    {
        return;
    }

    if (WheelSetups.Num() == 0)
    {
        PVehicle = nullptr;
        PVehicleDrive = nullptr;
```

```
        return;
    }

    for (int32 WheelIdx = 0; WheelIdx < WheelSetups.Num(); ++WheelIdx)
    {
        const FWheelSetup& WheelSetup = WheelSetups[WheelIdx];
        if (WheelSetup.BoneName == NAME_None)
        {
            return;
        }
    }

    // Setup the chassis and wheel shapes
    SetupVehicleShapes();

    // Setup mass properties
    SetupVehicleMass();

    // Setup the wheels
    PxVehicleWheelsSimData* PWheelsSimData = PxVehicleWheelsSimData::allocate(WheelSetups.Num());
    SetupWheels(PWheelsSimData);

    SetupVehicleDrive(PWheelsSimData);
}
```

4.2 CreateWheels

创建轮胎

C/C++

```cpp
void UWheeledVehicleMovementComponent::CreateWheels()
{
    // Wheels num is getting copied when blueprint recompiles, so we have to manually reset here
    Wheels.Reset();

    PVehicle->mWheelsDynData.setTireForceShaderFunction( PTireShader );

    // Instantiate the wheels
    for ( int32 WheelIdx = 0; WheelIdx < WheelSetups.Num(); ++WheelIdx )
    {
        UVehicleWheel* Wheel = NewObject<UVehicleWheel>( this, WheelSetups[WheelIdx].WheelClass );
        check(Wheel);

        Wheels.Add( Wheel );
    }

    // Initialize the wheels
    for ( int32 WheelIdx = 0; WheelIdx < Wheels.Num(); ++WheelIdx )
    {
        PVehicle->mWheelsDynData.setTireForceShaderData( WheelIdx, Wheels[WheelIdx] );

        Wheels[WheelIdx]->Init( this, WheelIdx );
    }
}
```

五、Tick（PhysX）

5.1 Substep

物理模拟大多数情况都是按照 Substep 来执行，Substep 是由 FPhysSubstepTask::StepSimulation 驱动

C/C++

```cpp
bool FPhysScene_PhysX::SubstepSimulation(FGraphEventRef &InOutCompletionEvent)
{
#if WITH_PHYSX
    float UseDelta = DeltaSeconds;
    float SubTime = PhysSubStepper->UpdateTime(UseDelta);
    PxScene* PScene = GetPxScene();
    if(SubTime <= 0.f)
    {
        return false;
    }else
    {
        //we have valid scene and subtime so enqueue task
        PhysXCompletionTask* Task = new PhysXCompletionTask(InOutCompletionEvent, PScene->getTaskManager(), &SimScratchBuffer);
        ENamedThreads::Type NamedThread = PhysSingleThreadedMode() ? ENamedThreads::GameThread : ENamedThreads::SetTaskPriority(ENamedThreads::GameThread, ENamedThreads::HighTaskPriority);

        DECLARE_CYCLE_STAT(TEXT("FSimpleDelegateGraphTask.SubstepSimulationImp"),
        STAT_FSimpleDelegateGraphTask_SubstepSimulationImp,
            STATGROUP_TaskGraphTasks);

        // 创建一个Task用于驱动PhysX的StepSimulation
        FSimpleDelegateGraphTask::CreateAndDispatchWhenReady(
            FSimpleDelegateGraphTask::FDelegate::CreateRaw(PhysSubStepper, &FPhysSubstepTask::StepSimulation, Task),
            GET_STATID(STAT_FSimpleDelegateGraphTask_SubstepSimulationImp), NULL, NamedThread
        );
        return true;
    }
#else
    return false;
#endif

}
```

C/C++

```
void FPhysSubstepTask::StepSimulation(PhysXCompletionTask * Task)
{
    check(SubTime > 0.f);
    check(DeltaSeconds > 0.f);

    FullSimulationTask = Task;
    Alpha = 0.f;
    StepScale = SubTime / DeltaSeconds;
    TotalSubTime = 0.f;
    CurrentSubStep = 0;

    SubstepSimulationStart();
}
```

真正物理模拟每次 Substep 的逻辑在 FPhysSubstepTask::SubstepSimulationStart 实现

C/C++

```
void FPhysSubstepTask::SubstepSimulationStart()
{
    SCOPE_CYCLE_COUNTER(STAT_TotalPhysicsTime);
    SCOPE_CYCLE_COUNTER(STAT_SubstepSimulationStart);
#if PHYSICS_INTERFACE_PHYSX
    check(SubTime > 0.f);
    check(DeltaSeconds > 0.f);

    check(!CompletionEvent.GetReference());    //should be done
    CompletionEvent = FGraphEvent::CreateGraphEvent();
    PhysXCompletionTask* SubstepTask = new PhysXCompletionTask(CompletionEvent,PAScene->getTaskManager());
    ENamedThreads::Type NamedThread = PhysSingleThreadedMode() ? ENamedThreads::GameThread : ENamedThreads::SetTaskPriority(ENamedThreads::GameThread, ENamedThreads::HighTaskPriority);

    DECLARE_CYCLE_STAT(TEXT("FDelegateGraphTask.ProcessPhysSubstepSimulation"),
        STAT_FDelegateGraphTask_ProcessPhysSubstepSimulation,
        STATGROUP_TaskGraphTasks);

    FDelegateGraphTask::CreateAndDispatchWhenReady(
        FDelegateGraphTask::FDelegate::CreateRaw(this, &FPhysSubstepTask::SubstepSimulationEnd),
        GET_STATID(STAT_FDelegateGraphTask_ProcessPhysSubstepSimulation), CompletionEvent, ENamedThreads::GameThread, NamedThread);

    ++CurrentSubStep;

    bool bLastSubstep = CurrentSubStep >= NumSubsteps;

    if (!bLastSubstep)
    {

        Alpha += StepScale;
        TotalSubTime += SubTime;
    }

```

```
    float DeltaTime = bLastSubstep ? (DeltaSeconds - TotalSubTime) : SubTime;
    float Interpolation = bLastSubstep ? 1.f : Alpha;

    // Call scene step delegate
#if !WITH_CHAOS
    if (PhysScene != nullptr)
    {
        FSubstepCallbackGuard Guard(*this);
        PhysScene->OnPhysSceneStep.Broadcast(PhysScene, DeltaTime);
    }
#endif

    SubstepInterpolation(Interpolation, DeltaTime);

#if WITH_APEX
    PAScene->simulate(DeltaTime, bLastSubstep, SubstepTask, FullSimulationTask->GetScratchBufferData(), FullSimulationTask->GetScratchBufferSize());
#else
    PAScene->lockWrite();
    PAScene->simulate(DeltaTime, SubstepTask, FullSimulationTask->GetScratchBufferData(), FullSimulationTask->GetScratchBufferSize());
    PAScene->unlockWrite();
#endif

    SubstepTask->removeReference();
#endif
}
```

Substep：

① **SubstepSimulationStart**

- CompletionEvent/SubstepTask

CompletionEvent：用于控制 Substep 循环调用

SubstepTask：PhysX 引擎层物理模拟调用任务，结束时会触发 CompletionEvent

- PhysScene->OnPhysSceneStep

PhysX 处理 Substep 前留给外部处理相关逻辑（例如物理输入），Vehicle 在引擎层处理的逻辑就在这里执行

- SubstepInterpolation

Substep 插值

- PAScene->simulate

PhysX 执行物理模拟

- SubstepTask->removeReference

② **SubstepSimulationEnd**

C/C++

```
void FPhysSubstepTask::SubstepSimulationEnd(ENamedThreads::Type CurrentThread, const FGraphEventRef& MyCompletionGraphEvent)
{
#if PHYSICS_INTERFACE_PHYSX
    CompletionEvent = NULL;
    // Substep未执行完
    if (CurrentSubStep < NumSubsteps)
    {
        uint32 OutErrorCode = 0;
        {
            SCOPE_CYCLE_COUNTER(STAT_TotalPhysicsTime);
            SCOPE_CYCLE_COUNTER(STAT_SubstepSimulationEnd);

            {
#if WITH_APEX
                SCOPED_APEX_SCENE_WRITE_LOCK(PAScene);
                PxScene* PScene = PAScene->getPhysXScene();
#else
                PxScene* PScene = PAScene;
                SCOPED_SCENE_WRITE_LOCK(PScene);
#endif

                // Dont rebuild scene queries here as this is an intermediate step
                PScene->setSceneQueryUpdateMode(PxSceneQueryUpdateMode::eBUILD_DISABLED_COMMIT_DISABLED);

                // 刷新物理状态
                PAScene->fetchResults(true, &OutErrorCode);

                // re-enable query updates (the next fetchResults will rebuild the SQ tree)
                PScene->setSceneQueryUpdateMode(PxSceneQueryUpdateMode::eBUILD_ENABLED_COMMIT_ENABLED);
            }
        }

        if (OutErrorCode != 0)
```

```
        {
            UE_LOG(LogPhysics, Log, TEXT("PHYSX FETCHRESULTS ERROR: %d"), OutErrorCode);
        }

        // 执行下一个Substep
        SubstepSimulationStart();
    }
    // Substep执行结束
    else
    {
        SCOPE_CYCLE_COUNTER(STAT_TotalPhysicsTime);
        SCOPE_CYCLE_COUNTER(STAT_SubstepSimulationEnd);

        //final step we call fetch on in game thread
        FullSimulationTask->removeReference();
    }
#endif
}
```

5.2 FPhysXVehicleManager::Update

C/C++

```cpp
void FPhysXVehicleManager::Update(FPhysScene* PhysScene, float DeltaTime)
{
    SCOPE_CYCLE_COUNTER(STAT_PhysXVehicleManager_Update);

    // Only support vehicles in sync scene
    if (Vehicles.Num() == 0 )
    {
        return;
    }

    if ( bUpdateTireFrictionTable )
    {
        SCOPE_CYCLE_COUNTER(STAT_PhysXVehicleManager_UpdateTireFrictionTable);
        bUpdateTireFrictionTable = false;
        // 更新轮胎摩擦力表
        UpdateTireFrictionTableInternal();
    }

    // Suspension raycasts
    {
        SCOPE_CYCLE_COUNTER(STAT_PhysXVehicleManager_PxVehicleSuspensionRaycasts);
        SCOPED_SCENE_READ_LOCK(Scene);
        // 轮胎射线检测
        PxVehicleSuspensionRaycasts( WheelRaycastBatchQuery, PVehicles.Num(), PVehicles.GetData(), WheelQueryResults.Num(), WheelQueryResults.GetData() );
    }


    // Tick vehicles
    {
        SCOPE_CYCLE_COUNTER(STAT_PhysXVehicleManager_TickVehicles);
        // 更新所有载具
        for (int32 i = Vehicles.Num() - 1; i >= 0; --i)
        {
```

```
            Vehicles[i]->TickVehicle(DeltaTime);
        }
    }

#if PX_DEBUG_VEHICLE_ON

    if ( TelemetryVehicle != NULL )
    {
        UpdateVehiclesWithTelemetry( DeltaTime );
    }
    else
    {
        UpdateVehicles( DeltaTime );
    }

#else

    UpdateVehicles( DeltaTime );

#endif //PX_DEBUG_VEHICLE_ON
    // Tick vehicles
    {
        SCOPE_CYCLE_COUNTER(STAT_PhysXVehicleManager_PostTickVehicles);
        for (int32 i = Vehicles.Num() - 1; i >= 0; --i)
        {
            Vehicles[i]->PostTick(DeltaTime);
        }
    }
}
```

5.3 UWheeledVehicleMovementComponent::TickVehicle

TG_StartPhysics 更新载具

C/C++

```
void UWheeledVehicleMovementComponent::TickVehicle( float DeltaTime )
{
    if (AvoidanceLockTimer > 0.0f)
    {
        AvoidanceLockTimer -= DeltaTime;
    }

    // movement updates and replication
    if (PVehicle && UpdatedComponent)
    {
        APawn* MyOwner = Cast<APawn>(UpdatedComponent->GetOwner());
        if (MyOwner)
        {
            UpdateSimulation(DeltaTime);
        }
    }

    // update wheels
    for (int32 i = 0; i < Wheels.Num(); i++)
    {
        Wheels[i]->Tick(DeltaTime);
    }

    UpdateDrag(DeltaTime);
}
```

5.3.1 UpdateSimulation

将输入数据传入物理引擎做后续的物理模拟

C/C++

```
void UWheeledVehicleMovementComponent4W::UpdateSimulation(float DeltaTime)
{
    if (PVehicleDrive == NULL)
        return;

    FBodyInstance* Instance = UpdatedPrimitive->GetBodyInstance();

    FPhysicsCommand::ExecuteWrite(Instance->GetActorReferenceWithWelding(), [&](const FPhysicsActorHandle& Actor)
    {
        PxVehicleDrive4WRawInputData RawInputData;
        RawInputData.setAnalogAccel(ThrottleInput);         // 油门
        RawInputData.setAnalogSteer(SteeringInput);         // 方向盘转向
        RawInputData.setAnalogBrake(BrakeInput);            // 刹车
        RawInputData.setAnalogHandbrake(HandbrakeInput);    // 手刹

        if(!PVehicleDrive->mDriveDynData.getUseAutoGears())
        {
            // 手动挡模式下的升档和降档
            RawInputData.setGearUp(bRawGearUpInput);
            RawInputData.setGearDown(bRawGearDownInput);
        }

        // 方向盘在不同速度下的最大转向比
        PxFixedSizeLookupTable<8> SpeedSteerLookup;
        TArray<FRichCurveKey> SteerKeys = SteeringCurve.GetRichCurve()->GetCopyOfKeys();
        const int32 MaxSteeringSamples = FMath::Min(8, SteerKeys.Num());
        for(int32 KeyIdx = 0; KeyIdx < MaxSteeringSamples; KeyIdx++)
        {
            FRichCurveKey& Key = SteerKeys[KeyIdx];
            SpeedSteerLookup.addPair(KmHToCmS(Key.Time), FMath::Clamp(Key.Value, 0.f, 1.f));
        }

        PxVehiclePadSmoothingData SmoothData = {
```

```
            { ThrottleInputRate.RiseRate, BrakeInputRate.RiseRate, HandbrakeInputRate.RiseRate, SteeringInputRate.RiseRate, SteeringInputRate.RiseRate },
            { ThrottleInputRate.FallRate, BrakeInputRate.FallRate, HandbrakeInputRate.FallRate, SteeringInputRate.FallRate, SteeringInputRate.FallRate }
        };

        PxVehicleDrive4W* PVehicleDrive4W = (PxVehicleDrive4W*)PVehicleDrive;
        PxVehicleDrive4WSmoothAnalogRawInputsAndSetAnalogInputs(SmoothData, SpeedSteerLookup, RawInputData, DeltaTime, false, *PVehicleDrive4W);
    });
}
```

5.3.2 TickWheel

C/C++

```
void UVehicleWheel::Tick( float DeltaTime )
{
    OldLocation = Location;
    Location = GetPhysicsLocation();
    Velocity = ( Location - OldLocation ) / DeltaTime;
}
```

5.3.3 UpdateDrag

C/C++

```cpp
void UWheeledVehicleMovementComponent::UpdateDrag(float DeltaTime)
{
    if (PVehicle && UpdatedPrimitive)
    {
        float ForwardSpeed = GetForwardSpeed();
        if (FMath::Abs(ForwardSpeed) > 1.f)
        {
            FVector GlobalForwardVector = UpdatedComponent->GetForwardVector();
            FVector DragVector = -GlobalForwardVector;
            float SpeedSquared = ForwardSpeed > 0.f ? ForwardSpeed * ForwardSpeed : -ForwardSpeed * ForwardSpeed;
            float ChassisDragArea = ChassisHeight * ChassisWidth;
            float AirDensity = 1.25 / (100 * 100 * 100); //kg/cm^3
            // 根据空气阻力方程计算空气阻力DragMag
            float DragMag = 0.5f * AirDensity * SpeedSquared * DragCoefficient * ChassisDragArea;
            DebugDragMagnitude = DragMag;
            DragVector *= DragMag;
            FBodyInstance* BodyInstance = UpdatedPrimitive->GetBodyInstance();
            BodyInstance->AddForce(DragVector, false);
        }

    }
}
```

六、Apply

应用物理效果：位移和动画

6.1 Move

应用物理位移，核心实现在 UPrimitiveComponent::SyncComponentToRBPhysics 函数中，在 TG_EndPhysics 阶段执行

- **调用堆栈**

```
USkeletalMeshComponent::MoveComponentImpl(const FVector &, const FQuat &, bool, FHitResult *, EMoveComponentFlags, ETeleportType) SkeletalMeshComponent.cpp:4083
USceneComponent::MoveComponent(const FVector &, const FRotator &, bool, FHitResult *, EMoveComponentFlags, ETeleportType) SceneComponent.cpp:2886
UPrimitiveComponent::SyncComponentToRBPhysics() PrimitiveComponentPhysics.cpp:597
USkeletalMeshComponent::EndPhysicsTickComponent(FSkeletalMeshComponentEndPhysicsTickFunction &) SkeletalMeshComponentPhysics.cpp:3190
FActorComponentTickFunction::ExecuteTickHelper<...>(UActorComponent *, bool, float, ELevelTick, const <lambda_93335ebd...> &) Actor.h:3581
FSkeletalMeshComponentEndPhysicsTickFunction::ExecuteTick(float, ELevelTick, Type, const TRefCountPtr<...> &) SkeletalMeshComponentPhysics.cpp:110
FTickFunctionTask::DoTask(Type, const TRefCountPtr<...> &) TickTaskManager.cpp:284
TGraphTask::ExecuteTask(TArray<...> &, Type) TaskGraphInterfaces.h:886
FNamedTaskThread::ProcessTasksNamedThread(int, bool) TaskGraph.cpp:735
FNamedTaskThread::ProcessTasksUntilQuit(int) TaskGraph.cpp:626
FTaskGraphImplementation::ProcessThreadUntilRequestReturn(Type) TaskGraph.cpp:1508
FTaskGraphImplementation::WaitUntilTasksComplete(const TArray<...> &, Type) TaskGraph.cpp:1559
FTickTaskSequencer::ReleaseTickGroup(ETickingGroup, bool) TickTaskManager.cpp:563
FTickTaskManager::RunTickGroup(ETickingGroup, bool) TickTaskManager.cpp:1577
UWorld::RunTickGroup(ETickingGroup, bool) LevelTick.cpp:814
UWorld::Tick(ELevelTick, float) LevelTick.cpp:1517
UGameEngine::Tick(float, bool) GameEngine.cpp:1806
FEngineLoop::Tick() LaunchEngineLoop.cpp:4926
EngineTick() Launch.cpp:62
GuardedMain(const wchar_t *) Launch.cpp:171
WinMain(HINSTANCE__ *, HINSTANCE__ *, char *, int) LaunchWindows.cpp:257
```

- **Code**

获取物理位置，如果发生了位移或旋转，移动 UpdatedComponent 以匹配物理状态

C/C++

```
void UPrimitiveComponent::SyncComponentToRBPhysics()
{
    if(!IsRegistered())
    {
        UE_LOG(LogPhysics, Log, TEXT("SyncComponentToRBPhysics : Component not registered (%s)"), *GetPathName());
        return;
    }

     // BodyInstance we are going to sync the component to
    FBodyInstance* UseBI = GetBodyInstance();
    if(UseBI == NULL || !UseBI->IsValidBodyInstance())
    {
        UE_LOG(LogPhysics, Log, TEXT("SyncComponentToRBPhysics : Missing or invalid BodyInstance (%s)"), *GetPathName());
        return;
    }

    AActor* Owner = GetOwner();
    if(Owner != NULL)
    {
        if (Owner->IsPendingKill() || !Owner->CheckStillInWorld())
        {
            return;
        }
    }

    if (IsPendingKill() || !IsSimulatingPhysics() || !RigidBodyIsAwake())
    {
        return;
    }

    // See if the transform is actually different, and if so, move the component to match physics
    const FTransform NewTransform = GetComponentTransformFromBodyInstance(UseBI);

```

```
    if(!NewTransform.EqualsNoScale(GetComponentTransform()))
    {
        const FVector MoveBy = NewTransform.GetLocation() - GetComponentTransform().GetLocation();
        const FRotator NewRotation = NewTransform.Rotator();

        //@warning: do not reference BodyInstance again after calling MoveComponent() - events from the move could have made it unusable (destroying the actor, SetPhysics(), etc)
        MoveComponent(MoveBy, NewRotation, false, NULL, MOVECOMP_SkipPhysicsMove);
    }
}
```

6.2 Animation

> 这里Demo的动画表现只用到了骨骼，因此下面主要以骨骼更新为例

- **获取轮胎的物理数据**

在 TG_PrePhysics 阶段，获取轮胎的物理数据（转动角度、转向角度、悬架弹簧压缩值）

> 这里之所以是在TG_PrePhysics阶段，其实可以理解为下一帧Tick的TG_PrePhysics，前一帧Tick会处理前面所有的任务：输入、物理模拟、移动等等，因此要等这些任务执行完毕后才能更新动画，引擎采取的方案是放在下一帧Tick的TG_PrePhysics阶段

C/C++

```cpp
void FVehicleAnimInstanceProxy::PreUpdate(UAnimInstance* InAnimInstance, float DeltaSeconds)
{
    Super::PreUpdate(InAnimInstance, DeltaSeconds);

    const UVehicleAnimInstance* VehicleAnimInstance = CastChecked<UVehicleAnimInstance>(InAnimInstance);
    if(const UWheeledVehicleMovementComponent* WheeledVehicleMovementComponent = VehicleAnimInstance->GetWheeledVehicleMovementComponent())
    {
        for (int32 WheelIndex = 0; WheelIndex < WheelInstances.Num(); ++ WheelIndex)
        {
            FWheelAnimData& WheelInstance = WheelInstances[WheelIndex];
            if (WheeledVehicleMovementComponent->Wheels.IsValidIndex(WheelIndex))
            {
                if (const UVehicleWheel* VehicleWheel = WheeledVehicleMovementComponent->Wheels[WheelIndex])
                {
                    // 获取轮胎的转动角度
                    WheelInstance.RotOffset.Pitch = VehicleWheel->GetRotationAngle();
                    // 获取轮胎的转向角度
                    WheelInstance.RotOffset.Yaw = VehicleWheel->GetSteerAngle();
                    WheelInstance.RotOffset.Roll = 0.f;

                    WheelInstance.LocOffset.X = 0.f;
                    WheelInstance.LocOffset.Y = 0.f;
                    // 获取轮胎悬架弹簧压缩值(相当于轮胎在纵向Z轴的偏移)
                    WheelInstance.LocOffset.Z = VehicleWheel->GetSuspensionOffset();
                }
            }
        }
    }
}
```

- **异步线程更新动画状态**

```
Thread-33-[TaskGraphThreadHP 26] (3988)
FAnimNode_WheelHandler::EvaluateSkeletalControl_AnyThread(FComponentSpacePoseContext &, TArray<...> &) AnimNode_WheelHandler.cpp:58
FAnimNode_SkeletalControlBase::EvaluateComponentSpace_AnyThread(FComponentSpacePoseContext &) AnimNode_SkeletalControlBase.cpp:175
FComponentSpacePoseLink::EvaluateComponentSpace(FComponentSpacePoseContext &) AnimNodeBase.cpp:442
FAnimNode_ConvertComponentToLocalSpace::Evaluate_AnyThread(FPoseContext &) AnimNodeSpaceConversions.cpp:35
FPoseLink::Evaluate(FPoseContext &) AnimNodeBase.cpp:356
FAnimNode_Root::Evaluate_AnyThread(FPoseContext &) AnimNode_Root.cpp:34
FAnimInstanceProxy::EvaluateAnimationNode_WithRoot(FPoseContext &, FAnimNode_Base *) AnimInstanceProxy.cpp:1465
FAnimInstanceProxy::EvaluateAnimation_WithRoot(FPoseContext &, FAnimNode_Base *) AnimInstanceProxy.cpp:1385
FAnimInstanceProxy::EvaluateAnimation(FPoseContext &) AnimInstanceProxy.cpp:1363
UAnimInstance::ParallelEvaluateAnimation(bool, const USkeletalMesh *, FParallelEvaluationData &) AnimInstance.cpp:756
USkeletalMeshComponent::EvaluateAnimation(const USkeletalMesh *, UAnimInstance *, FVector &, FBlendedHeapCurve &, FCompactPose &, FHeap
USkeletalMeshComponent::PerformAnimationProcessing(const USkeletalMesh *, UAnimInstance *, bool, TArray<...> &, TArray<...> &, FVector &, FB
USkeletalMeshComponent::ParallelAnimationEvaluation() SkeletalMeshComponent.cpp:3752
FParallelAnimationEvaluationTask::DoTask(Type, const TRefCountPtr<...> &) SkeletalMeshComponent.cpp:123
TGraphTask::ExecuteTask(TArray<...> &, Type) TaskGraphInterfaces.h:886
FTaskThreadAnyThread::ProcessTasks() TaskGraph.cpp:1106
FTaskThreadAnyThread::ProcessTasksUntilQuit(int) TaskGraph.cpp:927
FTaskThreadBase::Run() TaskGraph.cpp:565
FTaskThreadAnyThread::Run() TaskGraph.cpp:1005
FRunnableThreadWin::Run() WindowsRunnableThread.cpp:84
FRunnableThreadWin::GuardedRun() WindowsRunnableThread.cpp:27
FRunnableThreadWin::_ThreadProc(void *) WindowsRunnableThread.h:37
```

更新骨骼

C/C++

```
void FAnimNode_WheelHandler::EvaluateSkeletalControl_AnyThread(FComponentSpacePoseContext& Output, TArray<FBoneTransform>& OutBoneTransforms)
{
    check(OutBoneTransforms.Num() == 0);

    const TArray<FWheelAnimData>& WheelAnimData = AnimInstanceProxy->GetWheelAnimData();

    const FBoneContainer& BoneContainer = Output.Pose.GetPose().GetBoneContainer();
    for(const FWheelLookupData& Wheel : Wheels)
    {
        if (Wheel.BoneReference.IsValidToEvaluate(BoneContainer))
        {
            FCompactPoseBoneIndex WheelSimBoneIndex = Wheel.BoneReference.GetCompactPoseIndex(BoneContainer);

            // the way we apply transform is same as FMatrix or FTransform
            // we apply scale first, and rotation, and translation
            // if you'd like to translate first, you'll need two nodes that first node does translate and second nodes to rotate.
            FTransform NewBoneTM = Output.Pose.GetComponentSpaceTransform(WheelSimBoneIndex);

            FAnimationRuntime::ConvertCSTransformToBoneSpace(Output.AnimInstanceProxy->GetComponentTransform(), Output.Pose, NewBoneTM, WheelSimBoneIndex, BCS_ComponentSpace);

            // Apply rotation offset
            const FQuat BoneQuat(WheelAnimData[Wheel.WheelIndex].RotOffset);
            NewBoneTM.SetRotation(BoneQuat * NewBoneTM.GetRotation());

            // Apply loc offset
            NewBoneTM.AddToTranslation(WheelAnimData[Wheel.WheelIndex].LocOffset);

```

```
            // Convert back to Component Space.
            FAnimationRuntime::ConvertBoneSpaceTransformToCS(Output.AnimInstanceProxy->GetComponentTransform(), Output.Pose, NewBoneTM, WheelSimBoneIndex, BCS_ComponentSpace);

            // add back to it
            OutBoneTransforms.Add(FBoneTransform(WheelSimBoneIndex, NewBoneTM));
        }
    }
    ...
}
```

七、OnDestroyPhysicsState

C/C++

```
void UWheeledVehicleMovementComponent::OnDestroyPhysicsState()
{
    Super::OnDestroyPhysicsState();

    if ( PVehicle )
    {
        DestroyWheels();

        FPhysXVehicleManager* VehicleManager = FPhysXVehicleManager::GetVehicleManagerFromScene(GetWorld()->GetPhysicsScene());
        VehicleManager->RemoveVehicle( this );
        PVehicle = nullptr;
        PVehicleDrive = nullptr;

        if(MeshOnPhysicsStateChangeHandle.IsValid())
        {
            if(USkeletalMeshComponent* MeshComp = Cast<USkeletalMeshComponent>(GetMesh()))
            {
                MeshComp->UnregisterOnPhysicsCreatedDelegate(MeshOnPhysicsStateChangeHandle);
            }
        }

        if ( UpdatedComponent )
        {
            UpdatedComponent->RecreatePhysicsState();
        }
    }
}
```

八、Replication

内容较多，单独总结： Vehicle Replication